

Formelsammlung R

Teil B · ILIAS-Klausur am Computer · Statistik und Data Science I ·
Göttingen

★ Punkte-Retter

→ Rezept

Beispiel Schritt für Schritt

📄 Antwortsatz

⚠ Falle

TR Türkçe

Aufbau: Von den Grundlagen bis zu Likelihood und Tests – in der Reihenfolge, in der die Teilaufgaben in der Klausur typischerweise kommen.

Farbige Kästen: ★ gelb = Punkte-Retter · 📄 grün = Antwortsatz für ILIAS · ⚠ rot = typische Falle · → blau = Rezept · TR = Türkische Erklärung.

Code-Kästen: dunkler Hintergrund = R-Code; Zeilen mit # sind Kommentare, Zeilen mit [1] zeigen den Output.

Beispieldaten: Alle R-Outputs stammen aus `Beispiel_Mensa.csv` (Ordner uebungsdaten) – zum Nachrechnen.

Kapitel 15 enthält ein komplettes Musterskript für eine ganze Teil-B-Klausur – als Vorlage zum Abtippen und Anpassen.

Inhalt

0	Ablauf Teil B & Punkte-Retter	4
1	R-Grundlagen: Objekte, Vektoren, Indizes	5
2	Datentypen & Objekttypen	6
3	Working Directory & Daten einlesen	7
4	Data Frames bearbeiten	8
5	Deskriptive Statistik	9
6	Grafiken & als PDF speichern	10
7	Zusammenhänge: Tabellen, Korrelation, Regression	12
8	Verteilungen: d-, p-, q-, r-Funktionen	13
9	Simulation & Zufallszahlen	14
10	Likelihood & Maximum-Likelihood in R	15
11	Eigene Funktionen, Schleifen, Text	17
12	Kerndichteschätzer	18
13	Konfidenzintervalle & Tests in R	19
14	Fehlermeldungen & Rettung	20
15	Komplettes Musterskript einer Teil-B-Klausur	21
16	Antwortsatz-Vorlagen	22

So ist jede Teilaufgabe in ILIAS aufgebaut

- **Box 1: R-Code + R-Output** (aus der Konsole kopieren, mit `>` vor dem Code und dem `[1] ...` Output).
- **Box 2: inhaltliche Antwort** in vollen Sätzen, OHNE Code (nur wenn gefragt).
- Grafiken: als PDF speichern und hochladen; Titel mit `main` = Sitz, Matrikelnummer, Aufgabenteil; Achsen beschriften.
- Ergebnisse auf **3 Nachkommastellen** mit `round(..., 3)` runden – auch im Antwortsatz.
- Falschen Code, der nicht zur Antwort gehört, aus der ILIAS-Box löschen.

★ PUNKTE-RETTEN

Code immer abgeben – auch wenn er einen Fehler wirft. Ein richtiger Ansatz (z. B. `hist(d$Dauer, freq = FALSE, ...)`) bringt Teilpunkte.

Folgefehler werden berücksichtigt: Fehlt ein Objekt aus einer früheren Teilaufgabe, erzeuge es anders (z. B. Werte von Hand mit `c(...)`) und rechne weiter.

Datensatz lässt sich nicht einlesen? Trotzdem alle Befehle mit `d$Variable` hinschreiben – so steht es in den Hinweisen.

Antwortsatz = Frage wiederholen + Zahl mit 3 Nachkommastellen + Einheit/Sachzusammenhang.

Regelmäßig speichern (Skript + ILIAS)!

→ REZEPT

Arbeitsweise: 1) Neues R-Skript im Ordner Dokumente anlegen. 2) Jede Teilaufgabe als Abschnitt `## e)` im Skript. 3) Code mit Strg+Enter ausführen. 4) Code + Output aus der Konsole nach ILIAS kopieren. 5) Antwortsatz schreiben.

TR

Her alt soruda iki kutu var: (1) kod + çıktı, (2) cümleyle cevap. Hata alsan bile kodu yaz, yarım puan gelir. Sayıları `round(..., 3)` ile yuvarla.

```

x <- c(4, 8, 15, 16, 23, 42) # Vektor erzeugen (Zuweisung mit <-)
x[2]                        # 2. Element
[1] 8
x[c(1, 3)]                  # 1. und 3. Element
x[-1]                       # alle außer dem 1.
x[x > 10]                   # logische Auswahl
[1] 15 16 23 42
length(x); sum(x); prod(x)
seq(0, 1, by = 0.25)        # 0.00 0.25 0.50 0.75 1.00
seq(17, 23) # oder 17:23
rep(1, 5)                   # 1 1 1 1 1
sort(x); rev(x); order(x)  # sortieren, umdrehen, Reihenfolge-Index
x^2; sqrt(x); log(x); exp(x) # elementweise
which(x == 16)              # Position: 4
which.max(x); which.min(x)

```

Operator	Bedeutung
== !=	gleich / ungleich
< <= > >=	Vergleiche
& !	und / oder / nicht
%in%	ist enthalten in, z. B. d\$Fach %in% c("Jura", "Wiwi")
←	Zuweisung
#	Kommentar

⚠ FALLE

R unterscheidet Groß/Kleinschreibung: `d$Dauer` ≠ `d$dauer` . · Dezimalpunkt: 2.5, nicht 2,5. · Text in Anführungszeichen: "Jura".

Typ	Beispiel	class()	typeof()
ganze Zahl	3L, Spalte mit nur ganzen Zahlen aus read.csv	integer	integer
Kommazahl	2.5	numeric	double
Text	"Jura"	character	character
Kategorie	factor(c("a","b"))	factor	integer
Wahrheitswert	TRUE, FALSE	logical	logical
Tabelle	d ← read.csv(...)	data.frame	list
Matrix	matrix(1:6, nrow = 2)	matrix array	integer

```

class(d$Mensa)      # [1] "character"
class(d$Gerichte)   # [1] "integer"
typeof(d$Wartezeit) # [1] "double"
str(d)              # Überblick über alle Variablen und Typen
d$Mensa <- as.factor(d$Mensa) # in Faktor umwandeln
as.numeric("3.5")   # Text -> Zahl
levels(d$Mensa)     # Kategorien eines Faktors

```

🔗 ANTWORTSATZ

„Die Variable Mensa ist ein Objekt vom Typ character (Text), die Variable Gerichte ist ein numerisches Objekt vom Typ integer, d. h. sie enthält nur ganze Zahlen.“

★ PUNKTE-RETTEN

Bei „Objekttyp ermitteln“ immer einen Befehl (class/typeof/str) zeigen UND den Typ im Satz nennen – beides gibt Punkte.

3

Working Directory & Daten einlesen

```

setwd(path.expand("~/"))      # in der Klausur: Ordner Dokumente
getwd()                       # prüfen, wo man ist
list.files()                  # liegt die CSV-Datei hier?
d <- read.csv("Beispiel_Mensa.csv", sep = ",", dec = ".", header = TRUE)
# Semikolon-getrennt mit Dezimal komma:
d <- read.csv("Daten.csv", sep = ";", dec = ",") # oder read.csv2("Daten.csv")
d <- read.table("data.csv", sep = ";", header = TRUE)
head(d)                      # erste 6 Zeilen · head(d, 10) erste 10
tail(d); dim(d); nrow(d); ncol(d); names(d); summary(d)

```

→ REZEPT

Erst die CSV-Datei mit einem Texteditor ansehen: Trennzeichen „," oder „;“? Dezimal „.“ oder „,“? Kopfzeile vorhanden? → dann sep, dec, header passend setzen.

⚠ FALLE

Wenn head(d) nur EINE Spalte mit allen Werten zeigt, ist sep falsch. · Zahlen als character eingelesen → dec falsch. · Dateiname exakt (Groß-/Kleinschreibung, .csv).

```
d$Dauer           # eine Spalte (Vektor)
d[3, ]           # 3. Zeile
d[, 2]           # 2. Spalte
d[d$Mensa == "Nord", ] # nur Zeilen der Mensa Nord
subset(d, Typ == "Mitarbeiter" & Dauer > 40)
d$Stunden <- d$Dauer / 60 # neue Spalte
d$lang <- ifelse(d$Dauer > 40, "ja", "nein")
mean(d$Dauer[d$Typ == "Student"]) # Mittel einer Teilgruppe
sum(d$Typ == "Student")           # Anzahl Studierende (TRUE zählt 1)
mean(d$Typ == "Student")          # Anteil Studierende
d <- d[order(d$Dauer), ]          # nach Dauer sortieren
na.omit(d)                        # Zeilen mit NA entfernen
```

Kennzahl	R-Befehl	Achtung
arithm. Mittel	<code>mean(x)</code>	bei NA: <code>mean(x, na.rm = TRUE)</code>
Median	<code>median(x)</code>	
Modus	<code>names(which.max(table(x)))</code>	oder <code>table(x)</code> ansehen
Quantile	<code>quantile(x, c(0.25, 0.75))</code>	type = 7 Standard; Vorlesungsdefinition $\approx$ type = 2
Interquartilsabstand	<code>IQR(x)</code>	mit R-Standard (type 7)
Min, Max, Spannweite	<code>min(x), max(x), diff(range(x))</code>	
Varianz s^{*2} (n-1)	<code>var(x)</code>	unverzerrt!
Standardabw. s* (n-1)	<code>sd(x)</code>	unverzerrt!
Varianz s^2 (1/n)	<code>mean((x - mean(x))^2)</code> oder <code>var(x)*(n-1)/n</code>	falls „empirische Varianz“ verlangt
5-Punkte-Zusammenfassung	<code>summary(x)</code>	Min, Q1, Median, Mean, Q3, Max
Häufigkeiten	<code>table(x)</code>	absolut
relative Häufigkeiten	<code>prop.table(table(x))</code>	
kumuliert	<code>cumsum(prop.table(table(x)))</code>	
Gruppenmittel	<code>tapply(d\$Dauer, d\$Mensa, mean)</code>	oder <code>aggregate(Dauer ~ Mensa, data = d, FUN = mean)</code>
Runden	<code>round(x, 3)</code>	immer 3 Nachkommastellen

```
round(mean(d$Dauer), 3)
[1] 30.637
round(sd(d$Dauer), 3)
[1] 7.651
tapply(d$Dauer, d$Mensa, median)
  Nord  Turm Zentral
 31.5  29.0   33.0
round(quantile(d$Dauer, c(0.25, 0.5, 0.75)), 3)
```

📌 ANTWORTSATZ

„Das arithmetische Mittel der Variable Dauer beträgt 30.637. Der unverzerzte Schätzer für die Standardabweichung beträgt 7.651.“

⚠️ FALLE

`sd()` und `var()` teilen durch $n-1$ ($= s^*$). „Unverzerrte Standardabweichung“ $\rightarrow$ `sd()`. „Empirische Varianz mit $1/n$ “ $\rightarrow$ von Hand. · Gerundet nur das Endergebnis.

Rahmen zum Speichern (immer gleich)

```
pdf("Histogramm.pdf")      # Datei öffnen (im Working Directory)
... Grafikbefehl ...
dev.off()                  # Datei schließen – sonst ist das PDF leer!
```

Histogramm mit vorgegebenen Klassen

```
pdf("Histogramm.pdf")
hist(d$Dauer,
      breaks = c(0, 1, 2, 3, 5, 8), # Klassengrenzen
      right = FALSE,                # Intervalle [a, b): links geschlossen
      freq = FALSE,                 # y-Achse = Dichte
      main = "Sitz 12, Matr.Nr. 12345678, Aufgabenteil e)",
      xlab = "Aufenthaltsdauer in Stunden",
      ylab = "Dichte")
dev.off()
```

BEISPIEL – SCHRITT FÜR SCHRITT

Grafik-Teilaufgabe in 5 Schritten

Typische Aufgabe: „Erstellen Sie ein Histogramm ... Titel ... Achsen ... speichern Sie als Histogramm.pdf und laden Sie die Datei hoch.“

- 1 pdf("Histogramm.pdf") – Datei mit genau dem verlangten Namen öffnen.
- 2 Grafikbefehl mit allen Pflichtangaben: Daten, breaks, right = FALSE, freq = FALSE.
- 3 main = "Sitz ..., Matr.Nr. ..., Aufgabenteil e)", xlab und ylab exakt wie in der Aufgabe.
- 4 dev.off() – Datei schließen.
- 5 Code + Output nach ILIAS kopieren und die PDF-Datei hochladen.

Jeder Schritt ist ein eigener Bewertungspunkt – nichts weglassen!

⚠ FALLE

Ungleich breite Klassen → freq = FALSE (Dichte), sonst falsches Bild. · [a, b) → right = FALSE; (a, b] → right = TRUE (Standard). · Alle Daten müssen in den breaks liegen, sonst Fehler „some x not counted“.

Boxplot (auch nach Gruppen)

```
pdf("Boxplot.pdf")
boxplot(Dauer ~ Mensa, data = d,
        main = "Sitz 12, Matr.Nr. 12345678, Aufgabenteil e)",
        xlab = "Mensa", ylab = "Aufenthaltsdauer in Minuten")
dev.off()
boxplot(d$Dauer, horizontal = TRUE) # ein einzelner Boxplot
```

Weitere Grafiken

```
plot(d$Dauer, d$Wartezeit, xlab = "...", ylab = "...", main = "...") # Streudiagramm
abline(lm(Wartezeit ~ Minuten, data = d), col = "red")             # Regressionsgerade einzeichnen
barplot(table(d$Mensa), main = "...", xlab = "Mensa", ylab = "Anzahl")
plot(ecdf(d$Dauer), main = "...", xlab = "...", ylab = "F(x)")     # empirische Verteilungsfunktion
plot(density(d$Dauer), main = "...")                               # Kerndichteschätzer
curve(dnorm(x, 31, 8), from = 5, to = 60, add = TRUE, col = "blue") # Dichte darüber
lines(density(d$Dauer), col = "red")                               # Linie zu bestehendem Plot
legend("topright", legend = c("KDE", "Normal"), col = c("red", "blue"), lty = 1)
par(mfrow = c(1, 2))                                              # 2 Grafiken nebeneinander
```

★ PUNKTE-RETTEN

Titel und Achsenbeschriftung sind fast immer eigene Punkte: main = "Sitz ..., Matr.Nr. ..., Aufgabenteil ...", xlab, ylab GENAU wie in der Aufgabe.

PDF leer/fehlt? Grafik im Plot-Fenster über „Export → Save as PDF“ speichern – gibt ebenfalls Punkte.

```

tab <- table(d$Mensa, d$Typ)          # Kontingenztafel (absolut)
tab
addmargins(tab)                      # mit Randsummen
prop.table(tab)                      # relative Häufigkeiten (gesamt)
round(prop.table(tab, margin = 1), 3) # zeilenweise: Anteil je Mensa
round(prop.table(tab, margin = 2), 3) # spaltenweise

```

```

cov(d$x, d$y)                        # Kovarianz (n-1)
cor(d$x, d$y)                        # Pearson
cor(d$x, d$y, method = "spearman")  # Spearman (Rangkorrelation)
rank(d$x)                            # Ränge
modell <- lm(y ~ x, data = d)         # Regression  $y = a + b \cdot x$ 
coef(modell)                        # a (Intercept) und b
summary(modell)                     # inkl.  $R^2$  ("Multiple R-squared")
predict(modell, newdata = data.frame(x = 22)) # Prognose
round(summary(modell)$r.squared, 3)

```

🔗 ANTWORTSATZ

„Der Korrelationskoeffizient nach Bravais-Pearson beträgt 0.997; es liegt ein sehr starker positiver linearer Zusammenhang vor.“

„Die geschätzte Regressionsgerade lautet $\hat{y} = -8.654 + 3.269 \cdot x$. Steigt x um eine Einheit, steigt y im Mittel um 3.269 Einheiten.“

„In der Mensa Turm ist der Anteil der Mitarbeitenden mit 0.440 am höchsten.“

⚠️ FALLE

$\text{lm}(y \sim x)$: ZUERST die abhängige Variable (y), dann die erklärende (x). · margin = 1 → Zeilen, margin = 2 → Spalten. · cov/var in R mit n-1 – für r egal (kürzt sich).

8

Verteilungen: d-, p-, q-, r-Funktionen

Präfix	Bedeutung	Beispiel Normal
d...	Dichte bzw. $P(X = x)$	<code>dnorm(x, mean, sd)</code>
p...	Verteilungsfunktion $P(X \leq x)$	<code>pnorm(q, mean, sd)</code>
q...	Quantil: x mit $P(X \leq x) = p$	<code>qnorm(p, mean, sd)</code>
r...	Zufallszahlen	<code>rnorm(n, mean, sd)</code>

Verteilung	R-Name + Parameter	Beispiel
Binomial $B(n, \pi)$	<code>binom(x, size = n, prob = π)</code>	<code>dbinom(4, 5, 0.25)</code>
Poisson $Po(\lambda)$	<code>pois(x, lambda)</code>	<code>dpois(3, 2.5)</code>
Normal $N(\mu, \sigma^2)$	<code>norm(x, mean = μ, sd = σ)</code>	<code>pnorm(36, 30, 4)</code> ← sd, nicht Varianz!
Exponential $Exp(\lambda)$	<code>exp(x, rate = λ)</code>	<code>pexp(2, rate = 0.4)</code>
Gleichverteilung $U(a, b)$	<code>unif(x, min = a, max = b)</code>	<code>punif(0.3, 0, 1)</code>
t-Verteilung	<code>t(x, df)</code>	<code>qt(0.975, df = 8)</code>
χ^2 -Verteilung	<code>chisq(x, df)</code>	<code>qchisq(0.95, df = 3)</code>

Typische Wahrscheinlichkeiten

```

dbinom(4, size = 5, prob = 0.25)      # P(X = 4)
1 - pbinom(3, size = 5, prob = 0.25)  # P(X ≥ 4) = 1 - P(X ≤ 3)
pbinom(3, 5, 0.25, lower.tail = FALSE) # dasselbe: P(X > 3)
sum(dbinom(2:4, 5, 0.25))              # P(2 ≤ X ≤ 4)
ppois(2, lambda = 2.5)                 # P(X ≤ 2)
1 - pnorm(36, mean = 30, sd = 4)        # P(X > 36)
pnorm(35, 30, 4) - pnorm(25, 30, 4)    # P(25 ≤ X ≤ 35)
qnorm(0.975)                           # 1.959964
qt(0.95, df = 8)                       # 1.859548

```

⚠ FALLE

Diskret: $P(X \geq k) = 1 - \text{pbinom}(k - 1, \dots)$ ($k - 1!$). · Normal: sd = Wurzel aus der Varianz. · Exponential: Parameter heißt rate = λ (Erwartungswert $1/\lambda$).

9

Simulation & Zufallszahlen

```
set.seed(123) # reproduzierbar
x <- rnorm(100, mean = 50, sd = 10)
w <- sample(1:6, 1000, replace = TRUE) # Würfel 1000-mal
mean(w == 6) # relative Häufigkeit einer 6
sample(c("K", "Z"), 10, replace = TRUE)
mean(replicate(10000, mean(sample(1:6, 2, replace = TRUE)) > 4)) # Wahrscheinlichkeit per Simulation
```

a) Individuelle Likelihoods & Gesamt-Likelihood

```
d$iL02 <- dexp(d$Wartezeit, rate = 0.2) # stetig: Dichte
d$iL15 <- dpois(d$Gerichte, lambda = 1.5) # diskret: Wahrscheinlichkeitsfunktion
head(d)
L <- prod(d$iL02) # Likelihood = Produkt
L
[1] 1.690527e-89
format(L, scientific = TRUE, digits = 4) # 1.691e-89 -> 1.691 × 10(-89)
logL <- sum(log(d$iL02)) # Log-Likelihood = Summe der Logs
```

ANTWORTSATZ

„Unter der Annahme $\lambda = 0.2$ beträgt die Likelihood aller Beobachtungen $L(\lambda = 0.2) = 1.691 \times 10^{(-89)}$.“

FALLE

Darstellung $X \times 10^Y$: aus $1.690527e-89$ wird $1.691 \times 10^{(-89)}$ – X auf 3 Nachkommastellen, Y ganze Zahl (hier negativ!).

b) Likelihood-Funktion schreiben & über einem Gitter maximieren

```
Likelihood.exp <- function(lambda, x) {
  L <- prod(dexp(x, rate = lambda))
  return(L)
}
lam <- seq(0.15, 0.35, by = 0.05)
Ls <- sapply(lam, Likelihood.exp, x = d$Wartezeit)
cbind(lam, Ls) # Tabelle λ und L(λ)
lam[which.max(Ls)] # ML-Schätzer aus dem Gitter
[1] 0.2
# Alternative mit for-Schleife:
for (l in lam) print(Likelihood.exp(l, d$Wartezeit))
```

BEISPIEL – SCHRITT FÜR SCHRITT

Likelihood-Aufgabe in 4 Schritten

Poisson-Daten $x \leftarrow c(2, 0, 3, 1)$, Gitter $\lambda \in \{1, 1.5, 2\}$.

- 1 Individuelle Likelihoods: $\text{dpois}(x, \lambda = 1.5) \rightarrow 0.251 \ 0.223 \ 0.126 \ 0.335$.
- 2 Gesamt-Likelihood: $\text{prod}(\text{dpois}(x, 1.5)) = 0.002353 = 2.353 \times 10^{(-3)}$.
- 3 Funktion: $\text{Likelihood.pois} \leftarrow \text{function}(\lambda, x) \ \text{prod}(\text{dpois}(x, \lambda)); \text{sapply}(c(1, 1.5, 2), \text{Likelihood.pois}, x = x)$.
- 4 Maximum: $\text{which.max}(\dots) \rightarrow \lambda = 1.5$; Kontrolle analytisch: $\text{mean}(x) = 1.5 \checkmark$.

ML-Schätzer aus dem Gitter: $\hat{X} = 1.5$

c) Numerisch optimieren

```
logL <- function(lambda, x) sum(dexp(x, rate = lambda, log = TRUE))
optimize(logL, interval = c(0.01, 5), x = d$Wartezeit, maximum = TRUE) # $maximum =  $\lambda^{\wedge}$ 
# mehrere Parameter: negative Log-Likelihood mit nlm minimieren
nll <- function(p, x) -sum(dnorm(x, mean = p[1], sd = p[2], log = TRUE))
nlm(nll, p = c(30, 8), x = d$Dauer)$estimate
```

d) Analytische ML-Schätzer zum Vergleich

Modell	ML-Schätzer	R
Poisson(λ)	$\hat{\lambda} = \bar{x}$	mean(x)
Exponential(λ)	$\hat{\lambda} = 1/\bar{x}$	1/mean(x)
Bernoulli(π)	$\hat{\pi} = \text{Anteil}$	mean(x == "ja")
Normal μ, σ^2	$\hat{\mu} = \bar{x}, \hat{\sigma}^2 = (1/n)\sum(x-\bar{x})^2$	mean(x); mean((x-mean(x))^2)

🔗 ANTWORTSATZ

„Aus der vorgegebenen Menge maximiert $\lambda = 0.2$ die Likelihood; der ML-Schätzer ist daher $\hat{\lambda} = 0.2$. Der analytische ML-Schätzer $1/\bar{x} = 0.212$ liegt sehr nahe daran.“

★ PUNKTE-RETTEN

Funktion mit function(lambda, x) und return() sauber hinschreiben – auch ohne Ergebnis gibt die Struktur Punkte. Gitter mit apply ODER for-Schleife, beides ist erlaubt.

```
quadrat <- function(x) {           # Funktion definieren
  return(x^2)
}
quadrat(4)                         # [1] 16

for (i in 1:5) {                  # for-Schleife
  print(i^2)
}
erg <- numeric(5)                 # Ergebnisvektor vorbereiten
for (i in 1:5) erg[i] <- i^2

if (x > 0) { "positiv" } else { "nicht positiv" }
k <- 0; while (k < 3) { k <- k + 1 }
paste("Sitz", 12, "Aufgabe e)")   # "Sitz 12 Aufgabe e)"
paste0("0.", 1, 2, 3)             # "0.123" (ohne Leerzeichen)
paste(1:3, collapse = "")         # "123"
cat("Ergebnis:", round(3.14159, 3), "\n")
```

→ REZEPT

Aufgabe „Zahl aus aneinandergereihten Ziffern bauen“ (Typ Yennefer-Zahl):

```
f <- function(n) { s <- paste(1:n, collapse = ""); return(paste0("0.", s)) } → f(17) ergibt
"0.1234567891011121314151617"
```

```
dens <- density(d$Dauer)           # Standard: Gauß-Kern
plot(dens, main = "...", xlab = "Minuten", ylab = "Dichte")
density(d$Dauer, bw = 5)           # Bandweite festlegen
density(d$Dauer, kernel = "epanechnikov") # anderer Kern ("gaussian", "rectangular", "triangular",
"biweight")
hist(d$Dauer, freq = FALSE); lines(density(d$Dauer), col = "red")
```

- Große Bandweite → glattere Kurve, kleine Bandweite → zackige Kurve.
- „biweight“ entspricht dem Bisquare-Kern aus der Vorlesung.

t-Test (Erwartungswert, Varianz unbekannt)

```
t.test(d$Dauer, mu = 30, alternative = "greater", conf.level = 0.95)
```

```
One Sample t-test
data: d$Dauer
t = 0.74526, df = 79, p-value = 0.2292
alternative hypothesis: true mean is greater than 30
95 percent confidence interval:
 29.21378      Inf
sample estimates:
mean of x
 30.6375
```

Behauptung	H_1	alternative =
„weniger als μ_0 “	$\mu < \mu_0$	"less"
„mehr als μ_0 “	$\mu > \mu_0$	"greater"
„ungleich / unterscheidet sich“	$\mu \neq \mu_0$	"two.sided" (Standard)

```
t.test(x, mu = 200)$p.value           # nur p-Wert
t.test(x, conf.level = 0.95)$conf.int # zweiseitiges 95 %-KI
t.test(Dauer ~ Typ, data = d)         # Zwei-Stichproben-t-Test (Gruppen)
binom.test(x = 15, n = 20, p = 0.5, alternative = "greater") # Binomialtest
prop.test(x = 50, n = 80, p = 0.5)    # approx. Anteilstest / KI für  $\pi$ 
chisq.test(table(d$Mensa, d$Typ))     #  $\chi^2$ -Unabhängigkeitstest
chisq.test(c(20, 30, 50), p = c(0.25, 0.25, 0.5)) #  $\chi^2$ -Anpassungstest
```

KI von Hand (wenn verlangt)

```
n <- length(x); m <- mean(x); s <- sd(x)
m + c(-1, 1) * qt(0.975, df = n - 1) * s / sqrt(n) #  $\sigma$  unbekannt
m + c(-1, 1) * qnorm(0.975) * sigma / sqrt(n)      #  $\sigma$  bekannt
T <- (m - 70) / (s / sqrt(n)); T                   # Teststatistik von Hand
pt(T, df = n - 1)                                 # p-Wert linksseitig
```

ANTWORTSATZ

„Hypothesen: $H_0: \mu \leq 30$ vs. $H_1: \mu > 30$. Der p-Wert beträgt $0.229 > 0.05 = \alpha$. Daher kann H_0 nicht abgelehnt werden: Es kann nicht statistisch abgesichert werden, dass die mittlere Aufenthaltsdauer mehr als 30 Minuten beträgt.“

FALLE

Die Behauptung, die bewiesen werden soll, gehört in H_1 und bestimmt alternative. · „p-value = $2.2e-16$ “ heißt $p < 0.001$ (sehr klein). · conf.level bei einseitigem Test liefert ein einseitiges KI ($-\text{Inf}$...).

Fehlermeldung	Ursache	Lösung
cannot open file ... No such file	falsches Working Directory / Dateiname	getwd(), list.files(), setwd(path.expand("~")), Dateiname prüfen
object 'd' not found	Objekt noch nicht erzeugt	Zeile mit <code>d ← read.csv(...)</code> zuerst ausführen
could not find function	Tippfehler / Funktion nicht definiert	Schreibweise prüfen, eigene Funktion zuerst ausführen
non-numeric argument	Rechnen mit Text	<code>class()</code> prüfen, <code>as.numeric()</code> , <code>dec = ","</code> beim Einlesen
some 'x' not counted	Daten außerhalb der breaks	breaks so wählen, dass min und max abgedeckt sind
Ergebnis NA	fehlende Werte	<code>na.rm = TRUE</code>
unexpected symbol / '}'	Klammer oder Komma fehlt	Klammern zählen, Komma zwischen Argumenten
leeres PDF	<code>dev.off()</code> vergessen	<code>dev.off()</code> ausführen

★ PUNKTE-RETTET

Steckst du fest: nicht 10 Minuten an einem Fehler hängen. Code so abgeben, wie er ist (zeigt den Ansatz), und mit der nächsten Teilaufgabe weitermachen.

```
## a) Working Directory
setwd(path.expand("~/"))

## b) Einlesen + erste 6 Zeilen
d <- read.csv("Beispiel_Mensa.csv", sep = ",", dec = ".", header = TRUE)
head(d)

## c) Objekttypen
class(d$Mensa); class(d$Gerichte)

## d) Lage- und Streuungsmaße
round(median(d$Dauer), 3); round(IQR(d$Dauer), 3)
round(mean(d$Dauer), 3); round(sd(d$Dauer), 3)

## e) Grafik speichern
pdf("Boxplot.pdf")
boxplot(Dauer ~ Mensa, data = d, main = "Sitz 12, Matr.Nr. 12345678, Aufgabenteil e)",
        xlab = "Mensa", ylab = "Aufenthaltsdauer in Minuten")
dev.off()
tapply(d$Dauer, d$Mensa, median)

## f) Kontingenztafel + bedingte Anteile
tab <- table(d$Mensa, d$Typ); tab
round(prop.table(tab, margin = 1), 3)

## g) individuelle Likelihoods + Likelihood
d$il02 <- dexp(d$Wartezeit, rate = 0.2)
L <- prod(d$il02); L

## h) Likelihood-Funktion + Gitter
Likelihood.exp <- function(lambda, x) { return(prod(dexp(x, rate = lambda))) }
lam <- seq(0.15, 0.35, by = 0.05)
Ls <- sapply(lam, Likelihood.exp, x = d$Wartezeit)
cbind(lam, Ls); lam[which.max(Ls)]; 1 / mean(d$Wartezeit)

## i) Test
t.test(d$Dauer, mu = 30, alternative = "greater")
```

Teilaufgabe	Antwortsatz (Zahlen ersetzen)
Objekttyp	Die Variable Mensa ist vom Typ character, die Variable Gerichte vom Typ integer (ganze Zahlen).
Mittel / sd	Das arithmetische Mittel der Variable Dauer beträgt 30.637, der unverzernte Schätzer der Standardabweichung 7.651.
Median / IQR	Der Median der Aufenthaltsdauer beträgt 30.000 Minuten, der Interquartilsabstand 11.000 Minuten.
Grafik-Vergleich	In der Mensa Zentral ist der Median der Aufenthaltsdauer mit 33 Minuten am höchsten.
Gruppen	Gäste der Mensa ... bleiben im Durchschnitt am längsten (... Minuten).
Anteile	In der Mensa Turm ist der Anteil der Mitarbeitenden mit 0.440 am höchsten.
Likelihood	Die Likelihood aller Beobachtungen unter $\lambda = 0.2$ beträgt $1.691 \times 10^{(-89)}$.
ML-Gitter	Aus der vorgegebenen Menge maximiert $\lambda = 0.2$ die Likelihood; damit ist $\hat{\lambda} = 0.2$ der ML-Schätzer.
Korrelation	Der Korrelationskoeffizient beträgt ...; es liegt ein (schwacher/mittlerer/starker) (positiver/negativer) linearer Zusammenhang vor.
Regression	Steigt x um eine Einheit, so steigt/sinkt y laut Modell im Mittel um ... Einheiten.
Test ablehnen	Da der p-Wert (...) kleiner als $\alpha = 0.05$ ist, wird H_0 abgelehnt. Es kann statistisch abgesichert werden, dass ...
Test nicht ablehnen	Da der p-Wert (...) größer als $\alpha = 0.05$ ist, kann H_0 nicht abgelehnt werden. Es kann nicht statistisch abgesichert werden, dass ...
KI	Das 95 %-Konfidenzintervall für den Erwartungswert lautet [... ; ...].